ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Alliance School of Advanced Computing

Course Name: Graph Analytics

Course Code: E1ITC 302

Assignment No: 5

Name of the Student: B.Charitha sree

Registration Number: 2411021060285

A. Title of The Problem:

Graph Analytics – University Course Registration System Using Neo4j

B. Problem Statement in details:

A university requires a graph database to manage information about students, faculty members, courses, and academic departments. The system must represent these entities as nodes and their associations as relationships using Neo4j. It should maintain student enrolments, faculty teaching assignments, student–department associations, and department–course associations with appropriate properties. Cypher queries must be used to create the graph, retrieve relevant information, filter records based on specific conditions, and verify nodes and relationships. The system should enable efficient querying and analysis of the university's course registration data.

C. Student's Github link of report and code upload, along with the readme file:

https://github.com/reddycharitha04-ops/neo4j/upload

D. Solution Steps/Algorithm:

1. Open Neo4j Desktop, create a local database named UniversityDB, and start the database.
2. Create the Student, Faculty, Course, and Department nodes with their respective properties using Cypher queries.
3. Establish the relationships ENROLLED_IN, TEACHES, BELONGS_TO, and OFFERS with the specified directions.
4. Assign properties such as semesterEnrolled, grade, and academicYear to the relevant relationships.
5. Execute Cypher queries to display the complete graph, count nodes by label, and verify relationship types and directions.
6. Retrieve and filter information about students, courses, faculty members, and departments using the specified queries.
7. Verify the query outputs and capture screenshots of the graph, queries, and results for the assignment submission.

E. Quires:

1. Create all student, faculty, course, and department nodes with the given properties.

Query:

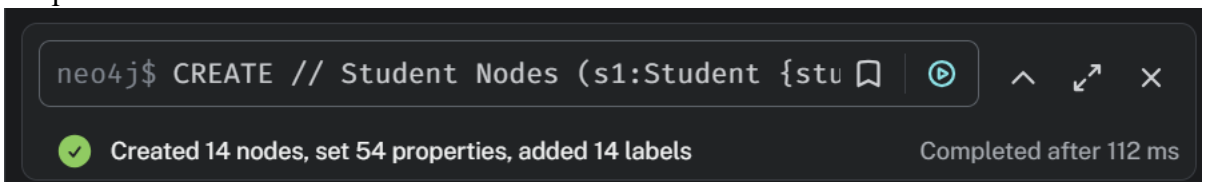
```
CREATE
// Student Nodes
(s1:Student {studentId: 'S101', name: 'Aarav Sharma', semester: 5, email: 'aarav@univ.edu'}),
(s2:Student {studentId: 'S102', name: 'Priya Das', semester: 5, email: 'priya@univ.edu'}),
(s3:Student {studentId: 'S103', name: 'Rahul Verma', semester: 3, email: 'rahul@univ.edu'}),
(s4:Student {studentId: 'S104', name: 'Sneha Roy', semester: 3, email: 'sneha@univ.edu'}),
(s5:Student {studentId: 'S105', name: 'Arjun Patel', semester: 5, email: 'arjun@univ.edu'}),

// Faculty Nodes
(f1:Faculty {facultyId: 'F101', name: 'Dr. Meera Sen', designation: 'Associate Professor',
specialization: 'Database Systems'}),
(f2:Faculty {facultyId: 'F102', name: 'Dr. Amit Roy', designation: 'Assistant Professor',
specialization: 'Data Analytics'}),

// Course Nodes
(c1:Course {courseCode: 'CS301', title: 'Graph Analytics', credits: 3, level: 'Advanced'}),
(c2:Course {courseCode: 'CS302', title: 'Database Management Systems', credits: 4, level:
'Core'}),
(c3:Course {courseCode: 'CS303', title: 'Data Mining', credits: 3, level: 'Advanced'}),
(c4:Course {courseCode: 'CS304', title: 'Big Data Analytics', credits: 4, level: 'Advanced'}),
(c5:Course {courseCode: 'CS305', title: 'NoSQL Databases', credits: 3, level: 'Core'}),

// Department Nodes
(d1:Department {departmentId: 'D101', name: 'Computer Science', building: 'Block A'}),
(d2:Department {departmentId: 'D102', name: 'Information Technology', building: 'Block
B'});
```

Output:



2. Create all the specified relationships with their appropriate directions.

```
 // 1. Student -> Course (ENROLLED_IN)
MATCH (s:Student), (c:Course)
WHERE
  (s.studentId = 'S101' AND c.courseCode IN ['CS301','CS302','CS305'])
  OR (s.studentId = 'S102' AND c.courseCode IN ['CS301','CS303'])
  OR (s.studentId = 'S103' AND c.courseCode IN ['CS302','CS304'])
  OR (s.studentId = 'S104' AND c.courseCode IN ['CS303','CS305'])
  OR (s.studentId = 'S105' AND c.courseCode IN ['CS301','CS304'])
CREATE (s)-[:ENROLLED_IN]->(c);

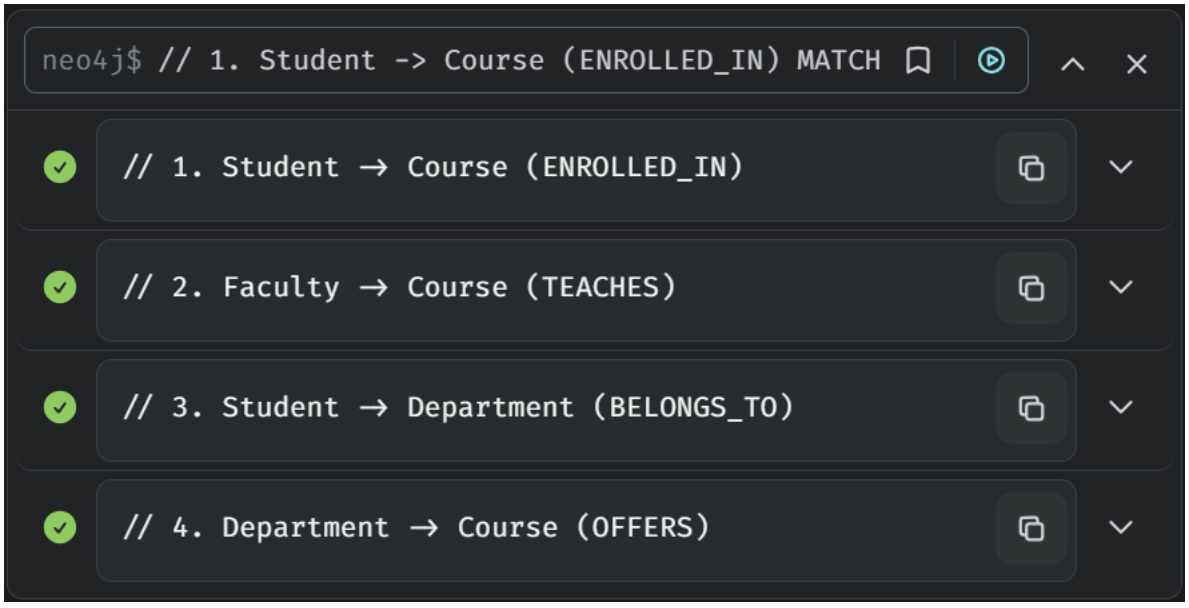
// 2. Faculty -> Course (TEACHES)
MATCH (f:Faculty), (c:Course)
WHERE
  (f.facultyId = 'F101' AND c.courseCode IN ['CS302','CS305'])
  OR (f.facultyId = 'F102' AND c.courseCode IN ['CS301','CS303','CS304'])
CREATE (f)-[:TEACHES]->(c);

// 3. Student -> Department (BELONGS_TO)
MATCH (s:Student), (d:Department)
WHERE
  (s.studentId IN ['S101','S102','S103']
    AND d.departmentId = 'D101')
  OR
  (s.studentId IN ['S104','S105']
```

```
   AND d.departmentId = 'D102')
CREATE (s)-[:BELONGS_TO]->(d);


// 4. Department -> Course (OFFERS)
MATCH (d:Department), (c:Course)
WHERE
  (d.departmentId = 'D101'
    AND c.courseCode IN ['CS302','CS303','CS305'])
  OR
  (d.departmentId = 'D102'
    AND c.courseCode IN ['CS301','CS304'])
CREATE (d)-[:OFFERS]->(c);
```

Output:



3. Assign properties to the ENROLLED_IN and TEACHES relationships.

Query:

A.ENROLLED_IN:

```
MATCH (s:Student)-[r:ENROLLED_IN]->(c:Course)
SET r.semesterEnrolled = s.semester,
```

r.grade = 'Not Published'

RETURN s.name AS Student,

c.title AS Course,

r.semesterEnrolled AS SemesterEnrolled,

r.grade AS Grade;

Output:



Student	Course	SemesterEnrolled	Grade
"Aarav Sharma"	"Graph Analytics"	5	"Not Published"
"Aarav Sharma"	"Graph Analytics"	5	"Not Publish
"Aarav Sharma"	"Database Management Systems"	5	"Not Published"
"Aarav Sharma"	"Database Management Systems"	5	"Not Published"
"Aarav Sharma"	"NoSQL Databases"	5	"Not Published"
"Aarav Sharma"	"NoSQL	5	"Not Published"

Set 110 properties — Started streaming 55 records after 215 ms and completed after 256 ms.

B. Teaches relationship:

Query:

MATCH (f:Faculty)-[r:TEACHES]->(c:Course)

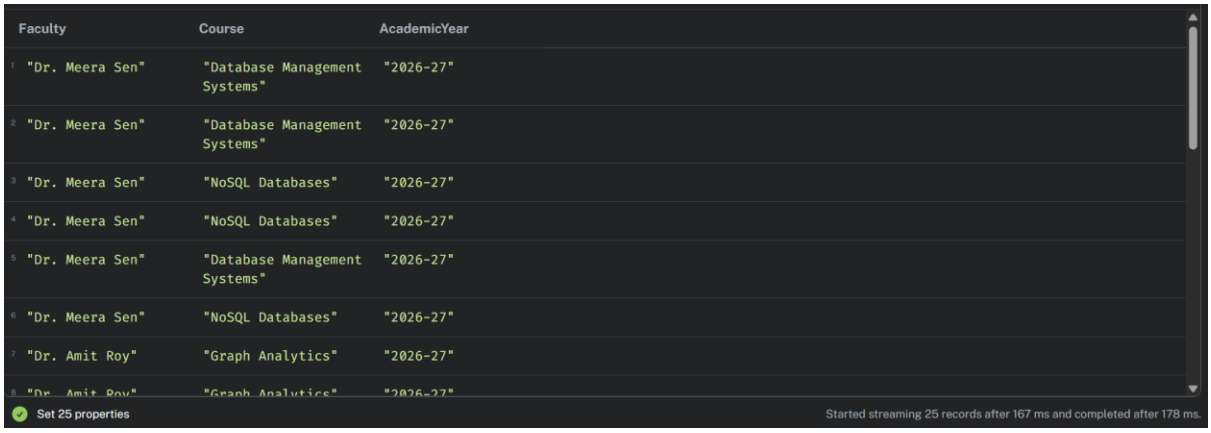
SET r.academicYear = '2026-27'

RETURN f.name AS Faculty,

c.title AS Course,

r.academicYear AS AcademicYear;

Output:



Faculty	Course	AcademicYear
"Dr. Meera Sen"	"Database Management Systems"	"2026-27"
"Dr. Meera Sen"	"Database Management Systems"	"2026-27"
"Dr. Meera Sen"	"NoSQL Databases"	"2026-27"
"Dr. Meera Sen"	"NoSQL Databases"	"2026-27"
"Dr. Meera Sen"	"Database Management Systems"	"2026-27"
"Dr. Meera Sen"	"NoSQL Databases"	"2026-27"
"Dr. Amit Roy"	"Graph Analytics"	"2026-27"
"Dr. Amit Roy"	"Graph Analytics"	"2026-27"

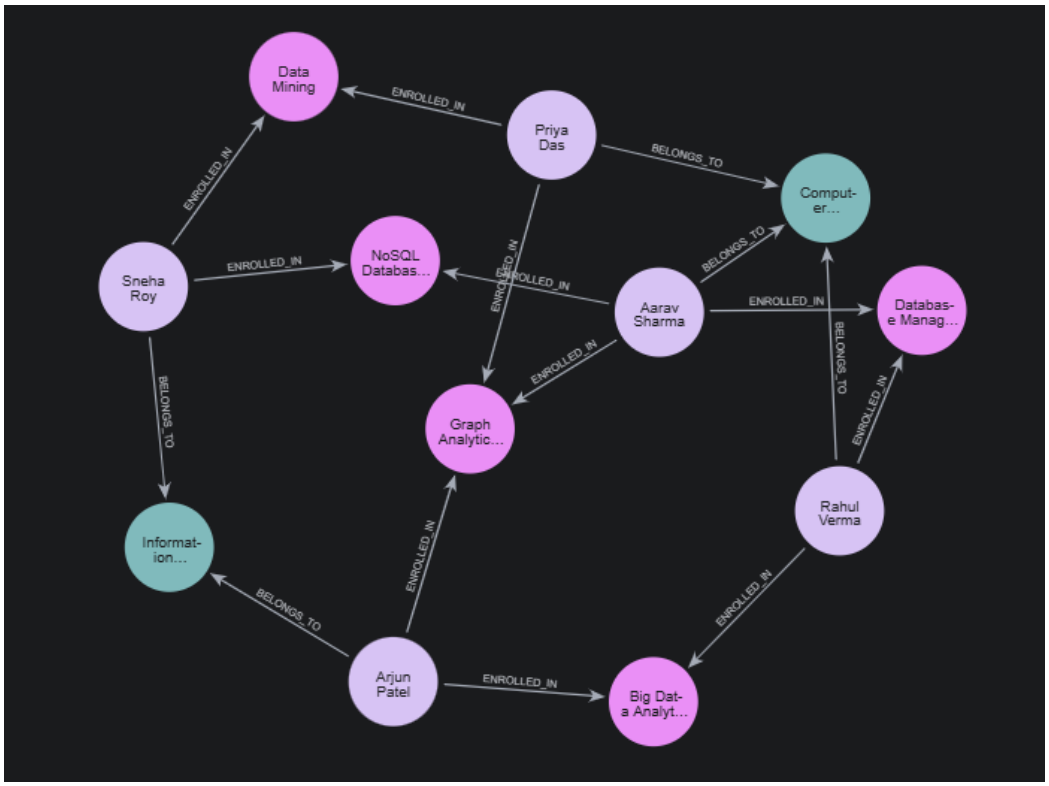
Set 25 properties — Started streaming 25 records after 167 ms and completed after 178 ms.

4. Display the complete graph.

Query:

```
MATCH (a)-[r]->(b)
RETURN a, r, b;
```

Output:



5. Count the number of nodes belonging to each label.

Query:

```
MATCH (n)
RETURN labels(n) AS NodeLabel,
       count(n) AS TotalNodes
ORDER BY NodeLabel;
```

Output:

	NodeLabel	TotalNodes
1	["Course"]	5
2	["Department"]	2
3	["Faculty"]	2
4	["Student"]	5

6. Display all relationship types and verify their directions.

Query:

```
MATCH (a)-[r]->(b)
RETURN labels(a) AS SourceLabel,
    type(r) AS RelationshipType,
    labels(b) AS TargetLabel,
    count(r) AS TotalRelationships
ORDER BY RelationshipType;
```

Output:

	SourceLabel	Relationship	TargetLabel	Total
1	["Student"]	"BELONGS_TO"	["Department"]	5
2	["Student"]	"ENROLLED_IN"	["Course"]	11
3	["Department"]	"OFFERS"	["Course"]	5
4	["Faculty"]	"TEACHES"	["Course"]	5

7. Retrieve all students studying in Semester 5

Query:

```
MATCH (s:Student)
WHERE s.semester = 5
RETURN s.studentId, s.name, s.semester, s.email;
```

Output:

	s.studentId	s.name	s.semester	s.email
1	"S101"	"Aarav Sharma"	5	"aarav@univ.edu"
2	"S102"	"Priya Das"	5	"priya@univ.edu"
3	"S105"	"Arjun Patel"	5	"arjun@univ.edu"

8. Find all courses with more than 3 credits.

Query:

```
MATCH (c:Course)
WHERE c.credits > 3
RETURN c.courseCode, c.title, c.credits;
```

Output:

	c.courseCode	c.title	c.credits
1	"CS302"	"Database Management Systems"	4
2	"CS304"	"Big Data Analytics"	4

9. Retrieve the courses taught by Dr. Meera Sen

Query:

```
MATCH (f:Faculty {name: 'Dr. Meera Sen'})
-[r:TEACHES]->(c:Course)
RETURN f.name AS Faculty, c.title AS Course,
r.academicYear AS AcademicYear;
```

Output:

	Faculty	Course	AcademicYear
1	"Dr. Meera Sen"	"Database Managem Systems"	"2026-27"
2	"Dr. Meera Sen"	"NoSQL Databases"	"2026-27"

10. Find all students enrolled in Graph Analytics.

Query:

```
MATCH (s:Student)-[r:ENROLLED_IN]->
(c:Course {title: 'Graph Analytics'})
RETURN s.name AS Student,
r.semesterEnrolled AS Semester,
r.grade AS Grade;
```

Output:

	Student	Semester	Grade
1	"Aarav Sharma"	5	"Not Published"
2	"Priya Das"	5	"Not Published"
3	"Arjun Patel"	5	"Not Published"

11. Display the names of all students belonging to the Computer Science department.

Query:

```
MATCH (s:Student)-[:BELONGS_TO]->
(d:Department {name: 'Computer Science'})
RETURN s.name AS Student, d.name AS Department;
```

Output:

	Student	Department
1	"Aarav Sharma"	"Computer Science"
2	"Priya Das"	"Computer Science"
3	"Rahul Verma"	"Computer Science"

12. Find all courses offered by the Information Technology department.

Query:

```
MATCH (d:Department {name: 'Information Technology'})
-[:OFFERS]->(c:Course)
RETURN d.name AS Department, c.courseCode, c.title;
```

Output:

	Department	c.courseCode	c.title
1	"Information Technology"	"CS301"	"Graph Analytics"
2	"Information Technology"	"CS304"	"Big Data Analytics"

13. Retrieve all students enrolled in NoSQL Databases along with their semester and grade.

Query:

```
MATCH (s:Student)-[r:ENROLLED_IN]->
(c:Course {title: 'NoSQL Databases'})
RETURN s.name AS Student,
s.semester AS CurrentSemester,
r.semesterEnrolled AS SemesterEnrolled,
r.grade AS Grade;
```

Output:

	Student	CurrentSemester	SemesterEnrolled	Grade
1	"Aarav Sharma"	5	5	"Not Published"
2	"Sneha Roy"	3	3	"Not Published"

14. Find all students enrolled in both Graph Analytics and another course.

Query:

```
MATCH (s:Student)-[:ENROLLED_IN]->
(g:Course {title: 'Graph Analytics'}),
(s)-[:ENROLLED_IN]->(other:Course)
WHERE other <> g
RETURN s.name AS Student,
collect(DISTINCT other.title) AS OtherCourses;
```

Output:

	Student	OtherCourses
1	"Aarav Sharma"	["Database Management Systems", "NoSQL Databases"]
2	"Priya Das"	["Data Mining"]
3	"Arjun Patel"	["Big Data Analytics"]

F.Conclusion:

The University Course Registration System was successfully implemented using Neo4j Desktop and the Cypher query language. Student, faculty, course, and department nodes were created with their respective properties. The required relationships were established with the specified directions, and properties were assigned to enrolment and teaching relationships. Retrieval and filtering queries were executed to obtain information about students, courses, faculty members, and departments. This assignment demonstrates how graph databases represent and query interconnected university data efficiently.